

# An EKF-based Multi-Object Tracking Framework for a Mobile Robot in a Precision Agriculture Scenario

Andrea Arlotta,

Martina Lippi,

Andrea Gasparri

**Abstract**—Many robotic applications require the ability to locate multiple objects in the environment, but the use of instant-by-instant identification techniques may be unreliable in variable and poorly structured contexts, such as for the majority of precision agriculture settings. Inspired by the needs of the H2020 CANOPIES projects, where robotic platforms are required to perform harvesting operations in table-grape vineyards, in this paper, we propose a framework for tracking objects of interest over time using a mobile robotic platform equipped with RGB-D camera. Specifically, we design a multi-object tracking module based on an Extended Kalman Filter (EKF) which takes into account the motion of the robot to update the estimate of the localization of the objects. We validate the approach in a realistic Unity-based simulator, where a mobile robot is tasked with tracking table-grape bunches within a vineyard environment. Additionally, we conduct preliminary tests in a laboratory setup.

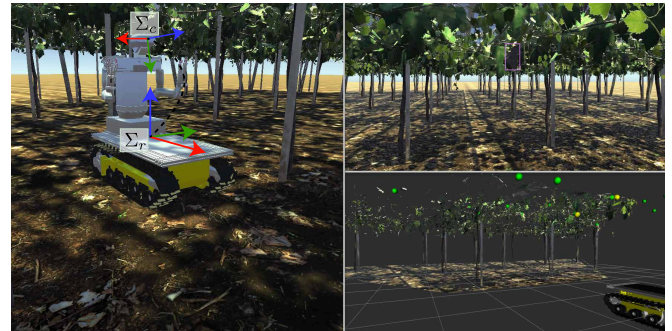
## I. INTRODUCTION

In many robotic applications, the ability to locate objects within the environment is a fundamental skill that enables various tasks such as monitoring and performing operations on such objects [1]–[3]. For instance, in autonomous vehicles, the ability to identify and localize other vehicles, pedestrians, and obstacles is crucial for safe navigation and collision avoidance. However, several application scenarios, such as in agricultural settings, are often characterized by complex, dynamical, and poorly structured environments, which can pose significant challenges for object detection algorithms, thus resulting in inadequate or unreliable performances. For example, occlusions, changes in lighting conditions, and other environmental factors can hinder the accuracy of object detection algorithms, leading to incorrect or incomplete identification of objects.

To overcome these challenges, multi-object tracking systems can be realized [4]. These techniques allow updating the estimate of the objects' location, even when they are temporarily occluded or move out of view. The classical approach for multi-object tracking systems relies on the use of cameras and tracking-by-detection paradigm [5], where an object detector is usually employed to detect the objects of interest and then a tracker is used to estimate their trajectories over time. One of the seminal works in this regard is [6] which relies on a multiple hypothesis tracking (MHT) approach. Specifically, this is based on creating multiple hypotheses for the identity and trajectory of each target, and then updating these hypotheses as new observations become available.

A. Arlotta, M. Lippi, and A. Gasparri are with the Department of Civil, Computer Science and Aeronautical Technologies Engineering, Roma Tre University, Italy, 00146 e-mail: [andrea.arlotta,martina.lippi@uniroma3.it](mailto:andrea.arlotta,martina.lippi@uniroma3.it), [gasparri@inf.uniroma3.it](mailto:gasparri@inf.uniroma3.it).

This work was supported by the European Commission under the grant agreement number 101016906 – Project CANOPIES.



**Fig. 1:** Representation of the case study within a precision agriculture context: the simulated vineyard with the robot (left), the RGB image as seen from the robot (top right) and the point cloud obtained from the robot RGB-D camera (bottom right) are reported.

The algorithm uses a Bayesian probabilistic framework to calculate the likelihood of each hypothesis given the current observations and previous hypotheses. Further examples of tracking-by-detection methods include SORT (Simple Online and Realtime Tracking) [7], DeepSORT [8], and IOU Tracker [9]. Regarding the object detector, many popular approaches can be used for tracking-by-detection, such as Fast Region-based Convolutional Neural Network (R-CNN) [10] or Faster R-CNN [11] as dual-stage detectors, and You Only Look Once (YOLO) [12] or Single Shot Multibox Detector (SSD) [13] as single stage detectors. More recently, another category of methods for multi-object tracking is emerging which is based on the tracking-by-regression paradigm, where the object's state is directly regressed from the image data. Examples of direct regression methods include the recent CenterTrack [14] and FairMOT [15]. However, these methods usually require large amounts of labeled training data and can be computationally expensive.

Despite the impressive results of recent methods for object detection and localization, they mainly rely on computer vision techniques and do not model the fact that the camera may be mounted on a mobile robot, whose motion should be taken into account for tracking. In this work, we design a tracking-by-detection framework for multi-object tracking using a mobile robot equipped with an RGB-D camera. Our framework employs an Extended Kalman Filter (EKF) for each tracked object, which is updated with velocity commands from the robot and instantaneous measurements. Given a detection module based on RGB-D data, we measure the relative positioning of the detected objects with respect to the robot. This instantaneous data is then processed through a data association procedure, which determines whether each measurement is associated with a tracked object or a new

one. Finally, the EKFs are updated using the respective measurements and the robot's velocity. This allows to address potential inaccuracies in the detection process due to, for instance, noise or occlusions. We validate the approach in a precision agriculture context, as shown in Figure 1, where the robot is deployed into a simulated realistic vineyard and tasked with locating bunches. Furthermore, we perform preliminary evaluations in a laboratory environment. The framework proposed in this work is a crucial step toward developing an autonomous mobile robot, with manipulation capabilities, for harvesting applications. By accurately detecting and tracking multiple objects, the robot can effectively execute trajectories to harvest ripe fruits, even in situations where they may be temporarily occluded by leaves or other fruits, or are outside the camera's field of view.

## II. PRELIMINARIES

### A. State and observation models

Let us consider a frame  $\Sigma_r$  attached to the mobile robot. We denote the frame where a variable is computed using the subscript index. If no frame is explicitly specified, we assume that the reference frame is  $\Sigma_r$ . Based on the requirements of the precision farming application of the H2020 project CANOPIES, which involves the detection of grape bunches to improve harvesting applications, we make the assumption that the objects of interest are static while the robot navigates within the environment. We denote the state of an object  $o$  by the vector  $p_o = [p_{x,o}, p_{y,o}, p_{z,o}]^T \in \mathbb{R}^3$ , which represents the object's position with respect to the robot frame. Thus,  $p_o$  denotes the location of the object  $o$  with respect to the robot. The evolution of the state variables over time is governed by nonlinear equations, which can be written as follows:

$$p_{o,k} = f(p_{o,k-1}, u_{k-1}) + w_{o,k-1}, \quad (1)$$

where  $p_{o,k}$  is the state of the object at time step  $k$ ,  $u_{k-1} \in \mathbb{R}^m$  is the *robot* control input at time step  $k-1$ ,  $f$  represents the state dynamics function relating the current state to the previous state and the control input, and  $w_{o,k-1} \in \mathbb{R}^3$  is the process noise at time step  $k-1$ . The latter is modeled as a zero-mean Gaussian process with covariance  $Q_{o,k-1} \in \mathbb{R}^{3 \times 3}$ . We assume to have noisy measurements of the object state. In detail, let  $z_{o,k} \in \mathbb{R}^3$  be the measurement vector of the object  $o$  at time  $k$ . The following observation model is considered:

$$z_{o,k} = h(p_{o,k}) + v_{o,k}, \quad (2)$$

where  $h(\cdot)$  is a nonlinear function that maps the state variables to the measurements, and  $v_{o,k} \in \mathbb{R}^3$  is a zero-mean Gaussian measurement noise with covariance  $R_{o,k} \in \mathbb{R}^{3 \times 3}$ . The specific state dynamics  $f(\cdot)$  and observation model  $h(\cdot)$  functions will be elaborated in the following sections with regard to the considered case study.

### B. Extended Kalman Filter

The Extended Kalman Filter (EKF) is a Bayesian filter commonly used for state estimation of systems with nonlinear dynamics and measurement models [16]. Briefly, it relies on linearizing the nonlinear dynamics around the current state

estimate and then propagating the state estimate forward in time using a recursive process. Two phases are recursively executed: 1) *Predict*, where the current state estimate and system dynamics in (1) are exploited to predict the next state estimate; 2) *Update*, where new measurements and the observation model in (2) are exploited to improve the state estimate. In detail, let  $\hat{x}_{o,k}$  be the object state estimate at time  $k$ , and  $P_{o,k} \in \mathbb{R}^{3 \times 3}$  the error covariance matrix associated with the state estimate. The prediction equations are

$$\begin{aligned} \hat{x}_{o,k}^- &= f(\hat{x}_{o,k-1}^-, u_{k-1}) \\ P_{o,k}^- &= F_{o,k-1} P_{o,k-1} F_{o,k-1}^T + Q_{o,k-1}, \end{aligned} \quad (3)$$

where  $\hat{x}_{o,k}^-$  represents the predicted state estimate at time  $k$ ,  $P_{o,k}^-$  is the predicted error covariance matrix, and  $F_{o,k-1}$  is the Jacobian matrix of the state model evaluated at  $\hat{x}_{o,k-1}$ . The update equations are

$$\begin{aligned} K_{o,k} &= P_{o,k}^- H_{o,k}^T (H_{o,k} P_{o,k}^- H_{o,k}^T + R_{o,k})^{-1} \\ \hat{x}_k &= \hat{x}_{o,k}^- + K_{o,k} (z_{o,k} - h(\hat{x}_{o,k}^-)) \\ P_{o,k} &= (I_3 - K_{o,k} H_{o,k}) P_{o,k}^- \end{aligned} \quad (4)$$

where  $H_{o,k}$  is the Jacobian matrix of the observation function evaluated at  $\hat{x}_{o,k}^-$ ,  $K_{o,k}$  is the Kalman gain and  $I_3$  is the  $3 \times 3$  identity matrix.

### C. Point cloud from RGB-D camera

RGB-D cameras are sensors that provide both color (RGB) and depth (D) information about a scene. This information can be used to generate a 3D point cloud, which represents the spatial locations of points in the scene. Each point in the point cloud corresponds to a pixel in the RGB image, and its position in 3D space is determined by the depth value at that pixel. More in detail, let  $\Sigma_c$  be the camera frame,  $f = [f_x, f_y]^T$  be the camera focal length,  $c = [c_x, c_y]^T$  be the principal point and  $s_d$  be the depth scale factor. We consider a generic pixel  $i$  with coordinates  $[u_i, v_i]^T$  and depth value  $d_i$ . The respective 3D point  $p_i^c = [x_i^c, y_i^c, z_i^c]^T$  in the camera frame is computed as follows:

$$\begin{bmatrix} x_i^c \\ y_i^c \\ z_i^c \end{bmatrix} = d_i s_d \begin{bmatrix} \frac{1}{f_x} & 0 & -\frac{c_x}{f_x} \\ 0 & \frac{1}{f_y} & -\frac{c_y}{f_y} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix}. \quad (5)$$

The overall camera point cloud  $\mathcal{P}^c$  is then formed by collecting all such  $p_i^c$  points associated with pixels having non-zero depth value  $d_i$ .

### D. Problem statement

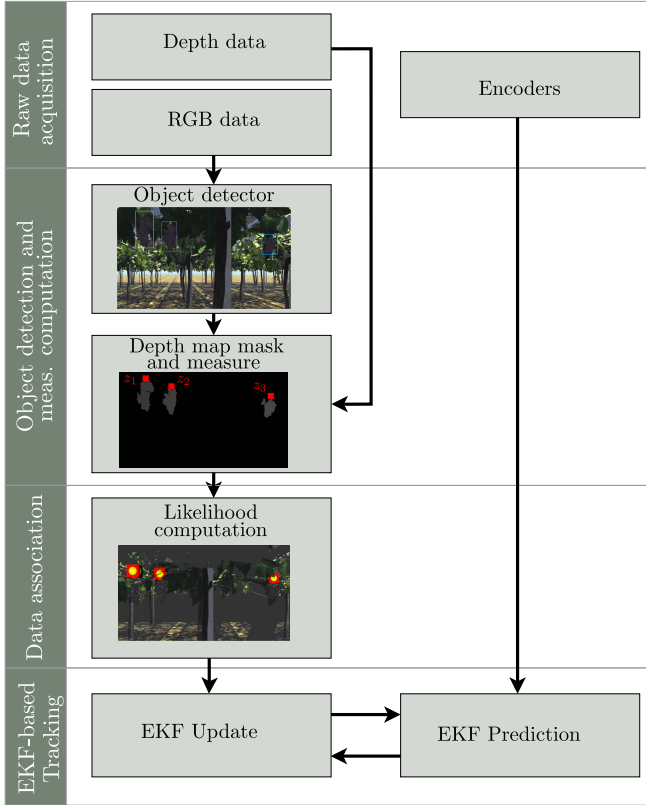
Based on the above quantities, we are now ready to state the main problem that we address in this work. Let us consider a mobile robot, with frame  $\Sigma_r$ , which is equipped with an RGB-D camera, with frame  $\Sigma_c$  and let  $T_c^r \in \mathbb{R}^{4 \times 4}$  be the homogeneous matrix of the camera frame with respect to the robot one. Let us consider that the robot navigates in an environment where  $n$  objects of interest are present, e.g., bunches in a table-grape vineyard. We aim to



**Fig. 2:** Examples of occlusions in a table-grape vineyard. On the left leaves occlude a bunch, on the right a black grape bunch occludes a white grape bunch behind.

develop a real-time framework for reliably estimating  $p_{o,k}$   $\forall o \in \{1, \dots, n\}$ , which represent the location of objects with respect to the robot frame, even when object measurements are intermittent or affected by outliers. We reiterate that such measurement inaccuracies are very frequent in unstructured outdoor environments, such as in precision agriculture settings. Figure 2 illustrates examples of occlusions in a table-grape vineyard. In the first scenario (left), a grape bunch (highlighted in a rectangle) is occluded by leaves. In the second scenario (right), a white grape bunch (highlighted in a rectangle) is occluded by a black grape bunch.

### III. PROPOSED FRAMEWORK



**Fig. 3:** Architecture overview.

The proposed framework to solve the above problem is represented in Figure 3 and structured into four main modules: *i)* raw data acquisition, *ii)* object detection and relative measurement computation, *iii)* data association, and *iv)* EKF for relative multi-object localization tracking. In the remainder of the paper, we omit the time dependency if not strictly necessary to simplify the notation. In the following, a description of each module is provided.

1) *Raw data acquisition:* This preliminary module, as illustrated by the first block on the left in Figure 3, gathers at each time step all the necessary data for multi-object tracking on the mobile robot. Specifically, encoder measurements are collected, which allow deriving the robot control input  $u_k$  in (3). This control input typically comprises the linear and angular velocities of the robot, as well as color and depth data from the RGB-D camera mounted on the robot.

2) *Object detection and relative measurement computation:* This module, shown in the second block from the left in Figure 3, is in charge of detecting the objects that are in the robot field of view at each time step  $k$  and providing the relative measurement vector  $z_{o,k}$  for each detected object  $o$ .

Regarding the object detector, we consider that any off-the-shelf object detector can be employed. For instance, in our prior works for precision agriculture settings such as [2], [3], [17], we employed the YOLO architecture, while, for the H2020 CANOPIES project, a grape cluster detection module was developed in [18] that is available for use. The detector produces two primary outputs: bounding boxes that enclose the objects of interest and a segmentation mask that identifies, for each detected object, the associated pixels. These outputs are leveraged to extract measurements for each object, which are then utilized for tracking purposes. As measurement function, we use

$$h(p_{o,k}) = p_{o,k}, \quad (6)$$

i.e., we measure the entire tracking vector. Measurements are obtained as follows. For each detected object  $o$ , the respective portion of the segmentation mask is used to obtain a *masked depth map*, such that zero values are assigned to the pixels not belonging to the object, while depth values are preserved for the object pixels. Let  $\mathcal{I}_o$  be the set of pixel indices with non-zero values in the masked depth map. The object point cloud  $\mathcal{P}_o^c$  is generated by applying (5) for the pixels in  $\mathcal{I}_o$ . At this point, to retrieve the measurement vector  $z_{o,k}^c$  in the camera frame, we select the pixel  $[u_o, v_o]^T$  at the center of the top edge of the bounding box that surrounds the object and approximate its depth value as  $\bar{d}_o = \frac{1}{|\mathcal{I}_o|} \sum_{i \in \mathcal{I}_o} d_i$ , where  $|\mathcal{I}_o|$  denotes the cardinality of  $\mathcal{I}_o$ . Then, the object measurement  $z_o^c$  is obtained by using  $u_o, v_o$ , and  $\bar{d}_o$  in (5). Note that the choice of the center of the top edge of the bounding box is arbitrary and other choices might be made depending on the task at hand. For instance, the center of the bounding box might be used. In our case study, the ultimate goal is to enable a robotic hand to execute harvesting, and this choice of bunch position allows us to more likely locate and target the bunch peduncle, that is the desired cutting point for the robotic hand. Additionally, we use the average depth value  $\bar{d}_o$  instead of the value  $d_o$  to calculate the object position because depth maps can be noisy, and the value  $d_o$  could even be zero. By utilizing this preprocessing step, we enhance the robustness of the system, and as we approach closer to the object of interest, more accurate perception data can be expected. As a final step, a coordinate transformation is applied to retrieve the object measurement  $z_o^r$  in the robot

frame. Specifically, it holds

$$\begin{bmatrix} z_o^r \\ 1 \end{bmatrix} = T_c^r \begin{bmatrix} z_o^c \\ 1 \end{bmatrix},$$

where  $T_c^r$  is the homogeneous matrix of the camera frame with respect to the robot one as stated in Section II-D. The measurements  $z_o^r$  obtained for all the objects detected at time step  $k$  are collected in the set of current measurements  $\mathcal{M}_k$ .

3) *Data association*: This module associates measurements with specific tracked objects in the environment. More in detail, given the set of tracked objects up to time  $k$   $\mathcal{T}_k$ , collecting the states of the tracked objects, and the set of current measurements  $\mathcal{M}_k$ , it enables us to establish whether the measurements are associated with any tracked object and if so, the specific corresponding one. To this aim, we compute a likelihood matrix  $L_k \in \mathbb{R}^{|\mathcal{M}_k| \times |\mathcal{T}_k|}$ , where each element  $l_{ij}$  represents the Euclidean distance between measurement  $i$ , i.e.,  $z_i$ , and tracked object  $j$ , i.e.,  $p_j$ . By computing the minimum likelihood value  $l_i^*$  in each row  $i$  of  $L_k$ , i.e.,  $l_i^* = \min_{j \in \mathcal{T}_k} l_{ij}$ , we can identify the best match between each new measurement  $i$  and the existing tracked objects. If the minimum value  $l_i^*$  is below a predefined likelihood threshold  $\tau$ , we preliminarily assign the new measurement  $i$  to the tracked object  $j^* = \operatorname{argmin}_{j \in \mathcal{T}_k} l_{ij}$ . If, on the other hand, the minimum value is above the threshold, we add the measure  $z_{i,k}$  to the set  $\mathcal{N}_k$ , collecting all the measures which need to be associated with new objects. Once all measurements have been analyzed, if multiple measurements are associated with the same object, we only preserve the association with minimum likelihood value. The third block from the left in Figure 3 illustrates the current module, showing the tracked objects in yellow and the measurements in red.

4) *EKF-based Multi-Object Tracking*: The objective of this module is to update the set of tracked objects  $\mathcal{T}$ . An EKF is associated with each tracked object, as reported in Section II-B. For each vector  $z_n \in \mathcal{N}_k$ , a new EKF is initialized with state  $p_{n,0} = z_n$  and the respective state is added to the set of tracked objects  $\mathcal{T}$ . Then, the prediction and update steps for each tracked object  $o$  are executed independently: the prediction step is triggered whenever a control input  $u_k$  is available, while the update step is triggered whenever a new measurement  $z_{o,k}$  is available. This enables the estimation of the relative object localization information even when a measurement is unavailable at a certain time  $k$ . To define the prediction model in (3), let us consider the expression of the object position with respect to a fixed world frame  $\Sigma_w$ , i.e.,

$$p_o^w = p_r^w + R_r^w p_o^r,$$

where  $p_r^w$  is the robot position with respect to frame  $\Sigma_w$ ,  $R_r^w$  is the rotation matrix between frame  $\Sigma_w$  and robot frame  $\Sigma_r$ , while  $p_o^r$  is the relative position of the object with respect to  $\Sigma_r$ . By deriving the previous equation, one obtains

$$\dot{p}_o^w = \dot{p}_r^w + R_r^w \dot{p}_o^r + \omega_r^w \times R_r^w p_o^r,$$

where  $\dot{p}_r^w$  and  $\omega_r^w$  denote the linear and angular velocities of the robot with respect to  $\Sigma_w$ , the operator  $\times$  denotes

the vector product, and the expression of the time derivative of rotation matrices has been exploited. By recalling the assumption made for our application scenario of having static objects, i.e.,  $\dot{p}_o^w = 0$ , the relative velocity of the object with respect to frame  $\Sigma_r$  can be computed as

$$\dot{p}_o^r = R_w^r [-\dot{p}_r^w - \omega_r^w \times R_r^w p_o^r].$$

At this point, by considering as frame  $\Sigma_w$  the frame of the robot at time step  $k-1$ , we derive the following dynamics

$$p_{o,k+1}^r = p_{o,k}^r - R_{r,k-1}^{r,k} (\dot{p}_{r,k}^{r,k-1} + \omega_{r,k}^{r,k-1} \times R_{r,k-1}^{r,k} p_{o,k}^r) \cdot \Delta t,$$

where  $\Delta t$  represents the sampling time,  $R_{r,k-1}^{r,k}$  expresses the rotation of the robot frame at time  $k-1$  with respect to the one at time  $k$ , and  $\dot{p}_{r,k}^{r,k-1}$  and  $\omega_{r,k}^{r,k-1}$  denote the linear and angular velocities of the robot at time  $k$  with respect to frame at time  $k-1$ . For instance, if the robot frame  $\Sigma_r$  is oriented with the x-axis aligned with the sagittal axis in the direction of motion (as shown in red in Figure 1) and the z-axis aligned with the vertical axis (in blue), the vector  $p_{r,k}^{r,k-1}$  have zero components along y and z axis, while the vector  $\omega_{r,k}^{r,k-1}$  will have zero components along x and y axis.

As far as the update phase is concerned, if a measurement in  $\mathcal{M}_k$  is associated with the object  $o$ , the update equations in (4), using measurement model in (6), are used.

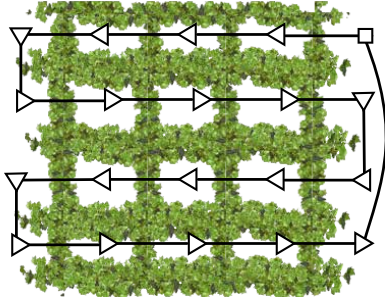
#### IV. VALIDATION RESULTS

To evaluate the effectiveness of the proposed approach, we firstly conducted a numerical validation within a simulated vineyard environment, purposely designed for the H2020 CANOPIES project, as depicted in Figure 1. Then, we performed a preliminary validation with a real robot in a laboratory setting. In both scenarios, the mobile robot was required to track table-grape bunches in its own (moving) frame. Validation results are shown in the video at the link<sup>1</sup>.

##### A. Setup description

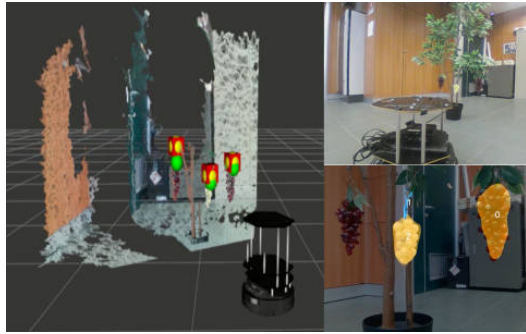
1) *Simulation setup*: The Unity-based simulation setup used in this study realistically simulates a vineyard with table-grape pergola system and  $3\text{m} \times 3\text{m}$  planting pattern. Different dimensions, grape densities, light orientation and intensity can be set to reproduce different operating conditions. Mobile robots, and in particular Alitrak DCT-300P mobile bases, can be included and equipped with a PAL Robotics humanoid robotic torso. Notably, the simulated environment, including the robotic platforms, is a reliable reproduction of the real-world one available for the experiments of CANOPIES project. Finally, the simulator generates ground truth data for object positions and is integrated with the Robot Operating System (ROS) middleware. In our experimental setup, we equipped the robot with an RGB-D camera mounted as shown in Figure 1. We validated the framework by having the robot navigate through the vineyard. Specifically, we created a grid of waypoints positioned at the center of each cell between the vines, as depicted in Figure 4 where triangles denote waypoints and the square denotes the start and final

<sup>1</sup><https://youtu.be/2s1Tz4dUjZ0>



**Fig. 4:** Example of path employed for testing tracking system. The square marker indicates the initial waypoint and the triangle markers indicate intermediate waypoints.

configuration. These waypoints were used to plan a path that ensured complete coverage of the field.



**Fig. 5:** Rviz environment screenshot from the laboratory validation session (left). View of the laboratory setup (top-right). Visual output of the object detection node (bottom-right).

2) *Laboratory setup:* Preliminary tests were conducted in the laboratory to evaluate the algorithm in a real environment, as shown in Figure 5. The setup included a synthetic tree with three artificial grape bunches and a Turtlebot 2 robot, equipped with a Realsense D435 RGB-D sensor. In addition, an Optitrack motion capture system was employed to obtain ground truth data. The robot was controlled using a joystick and moved in a random manner around the tree.

### B. Implementation details

The proposed framework has been fully realized within ROS middleware with the following main nodes:

*Robot odometry node*, receiving velocity commands for the mobile base as *Twist* messages and providing *Odometry* messages containing the state from the encoders.

*Trajectory generator node*, producing the desired waypoint as *PoseStamped* message.

*Robot controller*, producing velocity commands in the form of *Twist* messages based on the desired current waypoint.

*Camera node*, handling the RGB-D camera and publishing RGB and depth as *Image* messages. Concurrently, the camera intrinsic parameters are transmitted as *CameraInfo* messages.

*Object detection node*, performing the object detection and segmentation and publishing detected bounding boxes and segmentation mask via custom messages.

*Measurement computation node*, generating the measurements of the detected bunches with respect to the robot frame. It listens to the object detection node output and,

| Size | # Obj. | Mean [m] | Std. Dev. [m] | Min. [m] | Max. [m] | F.P. |
|------|--------|----------|---------------|----------|----------|------|
| 3x3  | 15     | 0.075    | 0.010         | 0.043    | 0.359    | 0    |
| 4x4  | 33     | 0.076    | 0.027         | 0.011    | 0.378    | 0    |
| 5x5  | 51     | 0.169    | 0.067         | 0.017    | 0.506    | 0    |
| 6x6  | 81     | 0.208    | 0.075         | 0.020    | 0.583    | 0    |

**TABLE I:** For each field dimension, the number of initialized filters, the values of the mean error, mean standard deviation, minimum error, maximum error, and number of false positives are reported.

when a detection message is available, it computes the set of 3D points associated to the bunches. The output of this computation is then published via a *MarkerArray* message, which provides the list of positions of detected bunches.

*Data association node*, responsible for possibly associating sensor measurements to tracked objects. Once the association process is complete, the measurements are tagged with an ID and published in a *MarkerArray* message.

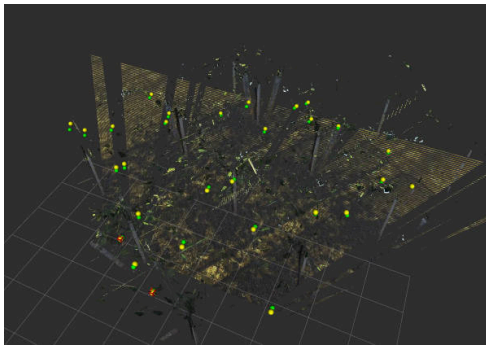
*Tracking node*, executing the EKFs associated with tracked objects. This node listens to the robot odometry topic and the output of the data association node: every time an odometry message is available, a prediction step is made for every object; while every time new measurements are available, a correction step is executed for the respective objects. The output of this computation provides a *MarkerArray* message which contains the set of the tracked objects.

Regarding the considered parameters, we set the measurement noise covariance matrix,  $R_{o,k} = \text{diag}\{0.5, 0.5, 0.5\}$ , and the process noise covariance matrix,  $Q_{o,k} = 10^{-4}I_3$ ,  $\forall o, k$ . Additionally, we initialized the error covariance matrix at time 0,  $P_{o,0} = 0.05I_3$ ,  $\forall o$ . The similarity threshold parameter for data association was set to  $\tau = 0.35$  m.

### C. Simulation results

We validated the framework with different field sizes, from 3 rows and 3 columns (denoted as  $3 \times 3$ ) to the case of 6 rows and 6 columns (denoted as  $6 \times 6$ ). Specifically, for each field size and for each waypoint provided by the trajectory generator, we collected the estimation errors, i.e., the distances, between ground truth data and estimated object positions through the EKFs. Table I summarizes the obtained results. Specifically, each row is associated with a field size, while columns report the number of instantiated EKFs, the average error, average standard deviation, minimum and maximum errors, and the number of False Positives (F.P., i.e., the number of active filters for the same object) obtained considering all waypoints. It can be observed that the mean and maximum error values increase with the size of the field. This is motivated by the fact that, according to our planned path, the higher the field size, the more likely it is that numerous bunches have had their latest measurements recorded long ago. Consequently, the estimates for these bunches are updated solely with predict equations in (3), resulting in an expected drift effect over time. Indeed, it is worth noting that the proposed approach is primarily well-suited for *local* multi-object tracking within the robot surroundings. The whole-field coverage simulation was merely conducted to showcase the results. In our future work, for instance, we intend to utilize the multi-object tracking system to guide the manipulator's movement for harvesting

applications. Therefore, only the information of the bunches in the vicinity of the robot are needed for an effective planning. In addition, the table reports a minimum error always greater than 0.01 m. This is motivated by the fact that, while the ground truth values correspond to the precise location of the peduncles of the bunches, we estimate their location by selecting the central points of the top edges of the bounding boxes associated with the detected bunches. This heuristic is based on the typical structure of grape bunches, where the peduncle is generally placed in the top-middle part of the bunch. Finally, the table shows that no false positives were recorded with the considered scenario.



**Fig. 6:** Top-view screenshot in RViz environment reporting the RGB-D depth cloud, measurements (red squares), ground truth data (green spheres), and estimated positions of the objects (yellow spheres).

Figure 6 shows a visual representation (generated in RViz) of the results obtained with the coverage of a vineyard composed of 3 rows and 3 columns. It reports the depth cloud generated by the RGB-D camera as well as the markers representing the ground truth positions of grape clusters (in green), the markers depicting poses generated by the measurement node (in red), and the markers obtained by the EKF of the tracked objects (in yellow). From the image, it can be observed that, in general, the EKF markers are relatively close to the ground truth markers. Certain EKF markers exhibit a more significant error, which can be attributed to the fact, as mentioned earlier, that the last measurement was obtained a long time ago, preventing the execution of the update equations.

#### D. Laboratory results

Figure 5 reports the laboratory setup (in the top right), the detection output, where the detected clusters are masked, (in the bottom right) as well as a graphical representation of the measurements, tracking instances, and ground truth (on the left). We reiterate that the detection module developed within the H2020 CANOPIES project, detailed in [18], was used. The robot moved for a total of 80 seconds. The figure, accompanied by the video, showcases the capabilities of the system to operate in a (simplified) real-world setup. Specifically, we can observe that the EKF markers (in yellow) closely approximate the ground truth markers (in green). On a quantitative basis, we recorded mean error value equal to 0.043 m with a standard deviation of 0.03 m, while maximum

and minimum errors were 0.148 m and 0.003 m, respectively. All bunches were successfully tracked with no false positives.

#### V. CONCLUSIONS

In this work, inspired by the needs of the H2020 CANOPIES project, we presented a framework for tracking table grape bunches in a vineyard using a mobile robotic platform equipped with an RGB-D camera. The proposed multi-object position tracking module is based on an Extended Kalman Filter which takes into account the robot motion for estimating the location of the objects of interest. As future work, we aim to validate the approach in a real setup and exploit the proposed framework for driving the motion of a mobile manipulator to carry out fruit harvesting operation, during which occlusion and clutter problems occur.

#### REFERENCES

- [1] R. Verschae and J. Ruiz-del Solar, "Object detection: current and future directions," *Frontiers in Robotics and AI*, vol. 2, p. 29, 2015.
- [2] M. Lippi, N. Bonucci, R. F. Carpio, M. Contarini, S. Speranza, and A. Gasparri, "A yolo-based pest detection system for precision agriculture," in *Mediterranean Conf. Control and Automation*, 2021, pp. 342–347.
- [3] A. Arlotta, M. Lippi, and A. Gasparri, "A ROS-based architecture for object detection and relative localization for a mobile robot with an application to a precision farming scenario," in *Mediterranean Conf. Control and Automation*, 2023.
- [4] W. Luo, J. Xing, A. Milan, X. Zhang, W. Liu, and T.-K. Kim, "Multiple object tracking: A literature review," *Artificial intelligence*, vol. 293, p. 103448, 2021.
- [5] Z. Sun, J. Chen, L. Chao, W. Ruan, and M. Mukherjee, "A survey of multiple pedestrian tracking based on tracking-by-detection framework," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 31, no. 5, pp. 1819–1833, 2020.
- [6] D. Reid, "An algorithm for tracking multiple targets," *IEEE Trans. Automatic Control*, vol. 24, no. 6, pp. 843–854, 1979.
- [7] N. Wojke, A. Bewley, and D. Paulus, "Simple online and realtime tracking with a deep association metric," in *IEEE Int. Conf. Image Processing*, 2017, pp. 3645–3649.
- [8] N. Wojke and A. Bewley, "Deep cosine metric learning for person re-identification," in *IEEE Winter Conf. Applications of Computer Vision*, 2018, pp. 748–756.
- [9] E. Bochinski, T. Senst, and T. Sikora, "Extending iou based multi-object tracking by visual information," in *IEEE Int. Conf. Advanced Video and Signals-based Surveillance*, 2018, pp. 441–446.
- [10] R. Girshick, "Fast R-CNN," in *IEEE Int. Conf. Computer Vision*, 2015, pp. 1440–1448.
- [11] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," *Advances in neural information processing systems*, vol. 28, 2015.
- [12] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," *arXiv preprint arXiv:2207.02696*, 2022.
- [13] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single shot multibox detector," in *Computer Vision—ECCV*. Springer, 2016, pp. 21–37.
- [14] X. Zhou, V. Koltun, and P. Krähenbühl, "Tracking objects as points," in *European Conf. Computer Vision*, 2020, pp. 474–490.
- [15] Y. Zhang, C. Wang, X. Wang, W. Zeng, and W. Liu, "Fairmot: On the fairness of detection and re-identification in multiple object tracking," *Int. J. Computer Vision*, vol. 129, pp. 3069–3087, 2021.
- [16] M. S. Grewal and A. P. Andrews, *Kalman filtering: Theory and Practice with MATLAB*. John Wiley & Sons, 2014.
- [17] M. Lippi, R. F. Carpio, M. Contarini, S. Speranza, and A. Gasparri, "A data-driven monitoring system for the early pest detection in the precision agriculture of hazelnut orchards," *IFAC-PapersOnLine*, vol. 55, no. 32, pp. 42–47, 2022.
- [18] T. A. Ciarfuglia, I. M. Motoi, L. Saraceni, M. Fawakherji, A. Sanfeliu, and D. Nardi, "Weakly and semi-supervised detection, segmentation and tracking of table grapes with limited and noisy data," *Computers and Electronics in Agriculture*, vol. 205, p. 107624, 2023.